# A Convolutional Neural Network for Object Detection

Daniel Petterson
300467629
Victoria University of Wellington
Wellington, New Zealand
petterdani@myvuw.ac.nz

***Abstract*—Reliably detecting objects in images has historically been a labour-intensive task. With the rise of Convolutional Neural Networks (CNN) it is increasingly simple to train a model to automatically detect objects at near or beyond the abilities of humans. This paper looks at how to build a CNN for detecting simple shapes and the effect that multiple objects in frame have on its accuracy.**

***Index Terms*—Convolutional Neural Network; Image Detection**

## Introduction

In the past decade the use of Convolutional Neural Networks has become widespread in computer vision tasks (Krizhevsky, Sutskever, and Hinton 2012) due to their ability to accurately detect patterns for object identification and image classification. The benefits of using a CNN over a fully connected multilayer perceptron (MLP) include mitigating the risk of overfitting in the event of insufficient data and being more memory efficient. Using MLPs for object detection would require a substantially larger network and significantly more data to be able to effectively learn variations in the image such as the orientation, location or size of objects (LeCun, Bengio, and others 1995).

## Proposed Approach

In this paper we will train a CNN with five hidden layers consisting of two convolutional layers, two pooling layers and a fully connected layer.

### *Network Architecture*

TABLE I: Layers and Output Dimensions

| Layer | Dimensions |
|---|---|
| Convolutional | 26x26x12 |
| MaxPooling | 13x13x12 |
| Convolutional | 11x11x12 |
| MaxPooling | 5x5x12 |
| Fully Connected | 300 |
| Fully Connected | 3 |

### *Convolutional Layer*

The convolutional layer takes input of dimensions (height, width, number of channels) and convolves them using kernels, a small matrix of weights. There is one kernel for each channel and together they make up a filter. These kernels are passed over the image and the values within the kernel boundaries are transformed based of the values of the kernel. The result of this is a feature map which can be thought of as a mapping that highlights certain aspects of the image such as edges or variations in colour. The feature maps are calculated using the following equation,

$$G[m,n] = (f * h)[m,n] = \sum_j \sum_k h[j,k] f[m-j, n-k] \tag{1}$$

where $f$ is the input image, $h$ is the kernel and $m$ and $n$ denote the rows and columns respectively of the resulting matrix. The weights of the kernels are what is learned during the training of a model. The dimensions of the output are defined as

$$Width = \frac{W - F_w + 2P}{S_w} + 1 \tag{2}$$

$$Height = \frac{H - F_h + 2P}{S_h} + 1 \tag{3}$$

where $W$ and $H$ are the width and height of the input image. $F$ denotes the filter, $P$ is a binary value to represent padding and $S$ is stride. The depth of the output represents the number of feature maps and is the same as the number of filters. Through activation, these feature maps contribute to the outcome prediction during training. We will be using the Rectified Linear Unit(ReLU) as our activation function for these layers. ReLU is a popular choice due to its resistance to the vanishing gradient effect in the backpropogation algorithm (Glorot, Bordes, and Bengio 2011). It creates sparse matrices through transforming any negative value to zero. The equation is as follows,

$$f(x) = max(0, x) \tag{4}$$

The benefits of this are two-fold, firstly it can significantly reduce the amount of memory required and secondly it may

result in faster processing of the data.

### *Pooling Layer*

A pooling layer acts to downsample the input and provide translation/shift invariance. The most common form of pooling is max-pooling, where the maximum value of the pooled area is chosen for transfer to the output. The goal of the pooling layer is similar to the convolutional layer in that they both aim to reduce dimensionality but pooling is much less computationally intensive at the expense of retention of resolution (Bochinski, Eiselein, and Sikora 2016). The resulting loss of resolution can be beneficial in reducing overfitting as the number of dimensions becomes smaller.

TABLE II: Result of 2x2 Max-Pooling

| | | | |
|---|---|---|---|
| **0** | **5** | 1 | 8 |
| **2** | **7** | 2 | 4 |
| 8 | 4 | 3 | 9 |
| 1 | 3 | 5 | 1 |

| | |
|---|---|
| **7** | 8 |
| 8 | 9 |

The above table shows the input and output of Max-Pooling where each subregion is a 2x2 matrix and the output is the maxmimum value of that subregion.

### *Fully Connected*

In transitioning from the previous layer to this one, the input is reshaped from three dimensions (width*height*depth) to a single dimension. As such this layer operates on a flattened input where each input is connected to all neurons and is a fully connected layer as used in common multilayer perceptrons.

### *Dataset Synthesis*

The image database is composed of 3,000 RGB images with a width and height of 28 pixels. Each image contains either a circle, triangle or rectangle at randomised positions but of identical size and orientation.

Fig. 1: Sample of Initial Images

### *Image Modification*

To evaluate how well this CNN structure performs on images that contain multiple objects, a circular arc or line was added to each image in the database.

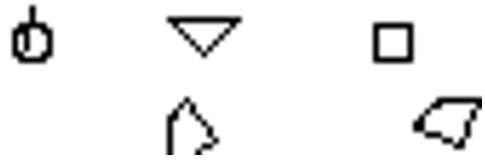

Fig. 2: Sample of Modified Images

## EXPERIMENTS

In order to assess the detection performance of the proposed method, experiments have been conducted on both initial and modified image databases. In each, the model was trained on a set of 1800 images and the performance was compared against a separate set of 600 images. The Stochastic Gradient Descent (SGD) solver was used, the initial weights were from a gaussian distribution with $\mu = 0$ and $\sigma = 0.05$. The training was done over 15 epochs with a learning rate of 0.01.

## RESULTS ON INITIAL SET OF IMAGES

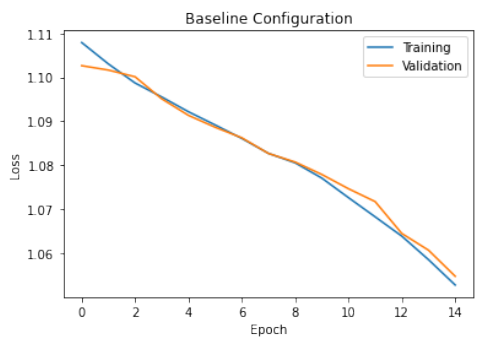


Fig. 3: Loss over Epochs

Looking at the scale we see only a minor reduction in the loss value over the training period.

## OPTIMISATION

In order to improve our model it is necessary to tune model hyperparameters. The four that we look at in this example are the optimiser used to set the model weights, the loss function which evaluates how far well the algorithm models the data, the learning rate of the optimiser and the size of the batches.

Over the training period neither the Stochastic Gradient Descent nor the Adagrad optimisation algorithms led to convergence whereas Adam and Root Mean Square Propagation(RMSProp) performed considerably better. It is notable that this performance changes when learning rate is modified and this difference in learning rate and how it changes is likely the reason for the poor performance of SGD. The learning rate in classical SGD is constant and applies to all weight updates similarly, RMSProp has a constant learning rate but this differs by parameter. In contrast to this the other two optimisation algorithms are able to dynamically alter the learning rates for each parameter over the course of training (Kingma and Ba 2017; Dogo et al. 2018). The two loss functions that

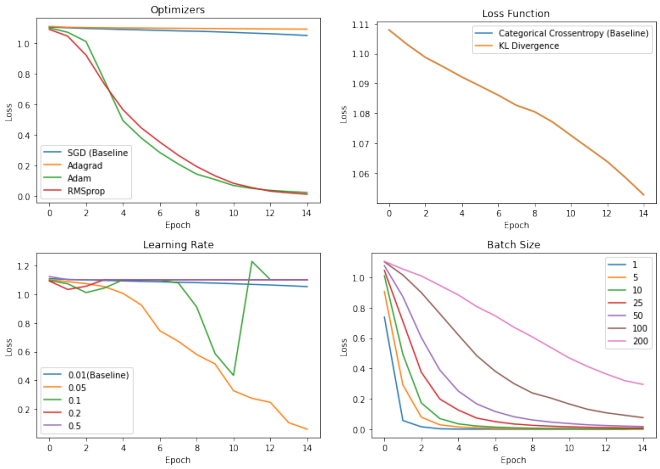


Fig. 4

we compared, Categorical Crossentropy and Kullback–Leibler Divergence performed nearly identically and a reduction in batch size is associated with more rapidly reducing loss but this reduction in batch size comes at significantly higher computational expense. As such we propose to change the optimisation algorithm of the CNN but retain the loss function and predefined batch size of 50 images.

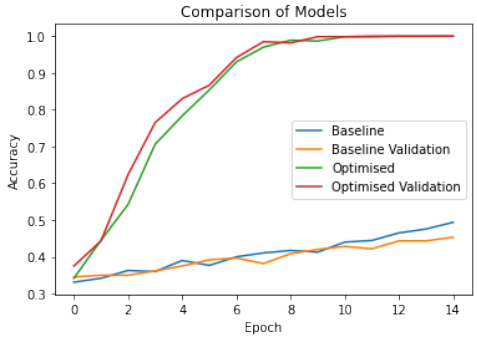


Fig. 5: Proportion of Correctly Detected Objects

There is a significant difference in the proportion of correctly classified objects between the initial model and the revised one which uses the Adam optimisation algorithm. The model achieved 100% accuracy on the test data and there does not appear to be any evidence of significant overfitting.

## Performance on Modified Image Database

For this task a model was trained with the previous hyperparamters but over fifth epochs instead of fifteen. It is reasonable to assume that it would require more adjustments of weights since the signal is less clear than on the initial set of images.

While the model managed to correct identify objects in 89.8% of test images the divergence in accuracy performance between the training and validation image sets suggests overfitting. There are many ways to reduce overfitting, the simplest of which would be to generate more training data to allow the

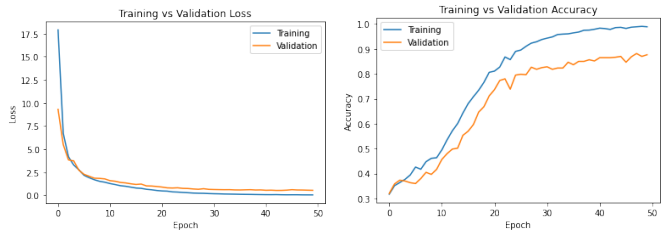


Fig. 6: Training Performance on Images with Multiple Objects

model to better generalise to unseen images. We could also use regularisation in the form of a dropout layer which randomly sets activations to zero during the training process. Another option is to reduce model complexity by reducing the number of hidden layers in the model. We will incorporate the latter two options into our CNN design as well as increasing the filter size from three to five in order to slightly mitigate the resulting increase in dimensionality.

## The Revised Model

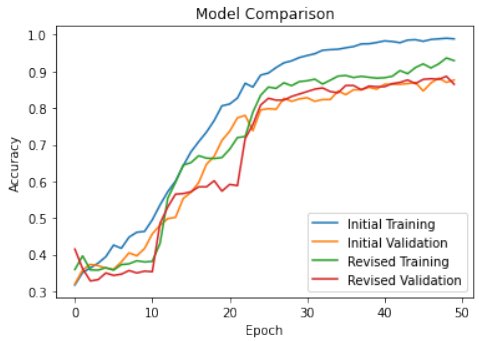


Fig. 7: Revised Model Performance Compared to Initial Model

The effect of overfitting is less pronounced than previously but the accuracy on the test set is similar at 87%. For further performance improvements it would be prudent to generate more data for use in training so that the model can handle variations in patterns caused by the randomised addition of lines and shapes.

## APPENDIX

The accompanying code can be found in a Colab Notebook at the following link: Google Colab Notebook

## REFERENCES


Bochinski, Erik, Volker Eiselein, and Tomas Sikora. 2016. "Training a Convolutional Neural Network for Multi-Class Object Detection Using Solely Virtual World Data." In *2016 13th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, 278–85. IEEE.

Dogo, EM, OJ Afolabi, NI Nwulu, B Twala, and CO Aigbavboa. 2018. "A Comparative Analysis of Gradient Descent-Based Optimization Algorithms on Convolutional Neural Networks." In *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*, 92–99. IEEE.

Glorot, Xavier, Antoine Bordes, and Yoshua Bengio. 2011. "Deep Sparse Rectifier Neural Networks." In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 315–23. JMLR Workshop; Conference Proceedings.

Kingma, Diederik P., and Jimmy Ba. 2017. "Adam: A Method for Stochastic Optimization." http://arxiv.org/abs/1412.6980.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E Hinton. 2012. "Imagenet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems* 25: 1097–1105.

LeCun, Yann, Yoshua Bengio, and others. 1995. "Convolutional Networks for Images, Speech, and Time Series." *The Handbook of Brain Theory and Neural Networks* 3361 (10): 1995.